

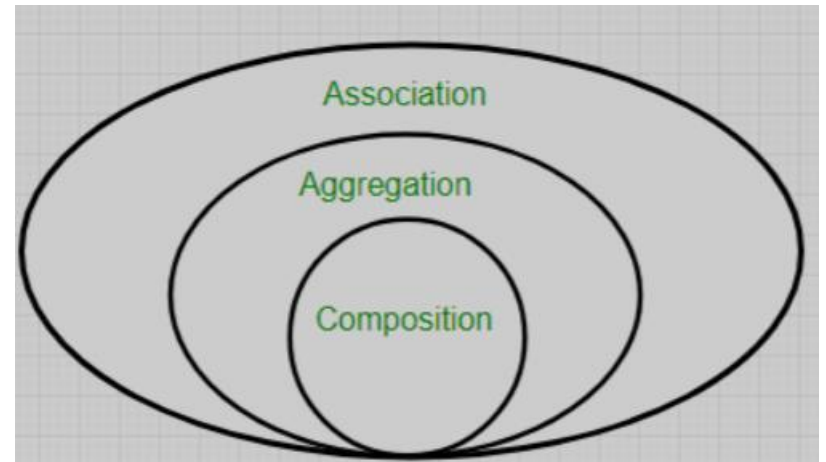
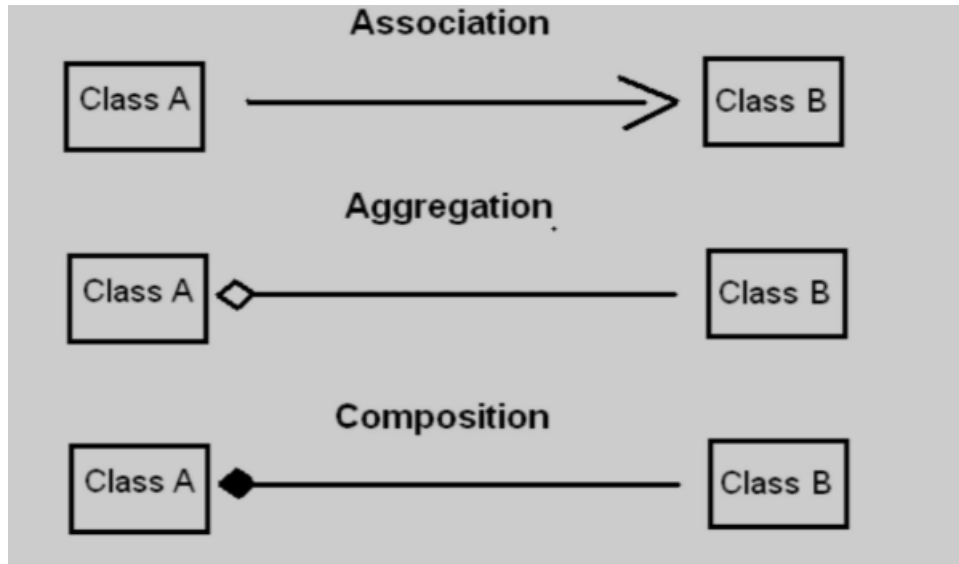
Object-oriented Programming

Week 4

Object Relationship

- One of the advantages of an Object-Oriented programming language is code reuse.
- There are two ways we can do code reuse either by the implementation of inheritance (IS-A relationship), or object composition (HAS-A relationship).

UML notations



Object Relationship

- Often, classes do not have to be created “from scratch.” Rather, they can include objects of other classes as members or they may be **derived** from other classes that provide attributes and behaviors the new classes can use. Such software reuse can greatly enhance productivity and simplify code maintenance. Including class objects as members of other classes is called **composition** (or **aggregation**). Deriving new classes from existing classes is called **inheritance**

Object Relationship

- is-a relationship (Achieved by inheritance)
- has-a relationship (Achieved by composition)

has-a Relationship

- In a *has-a* relationship, an object contains one or more objects of other classes as members

A technique in which a class contains an object of another class as its member is called containership. This type of relationship between classes is also called *has-a* relationship. It means that one class has an object of another class as its member.

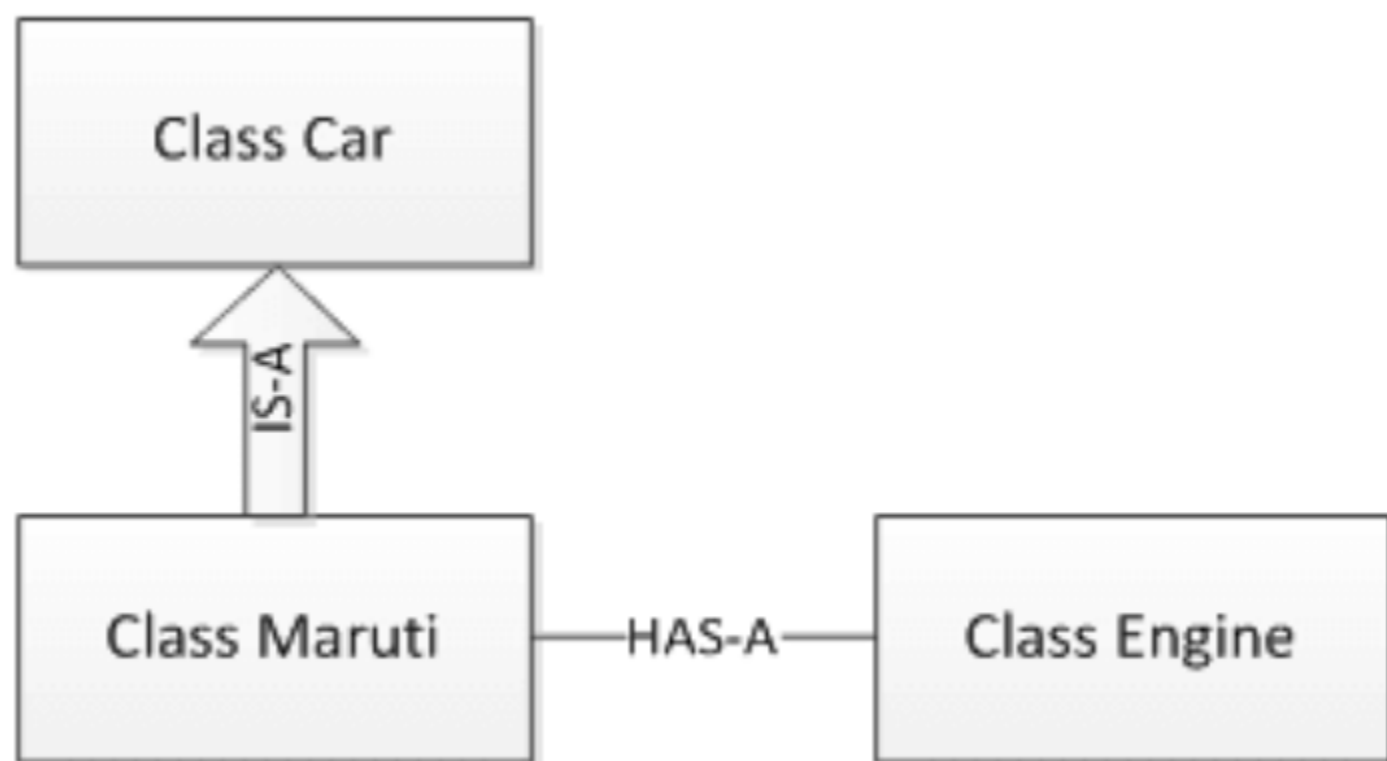
- “**Has-A**” — An object **has** a subset of object
- For example is :-
- A rose has a leaf
- A dog has legs

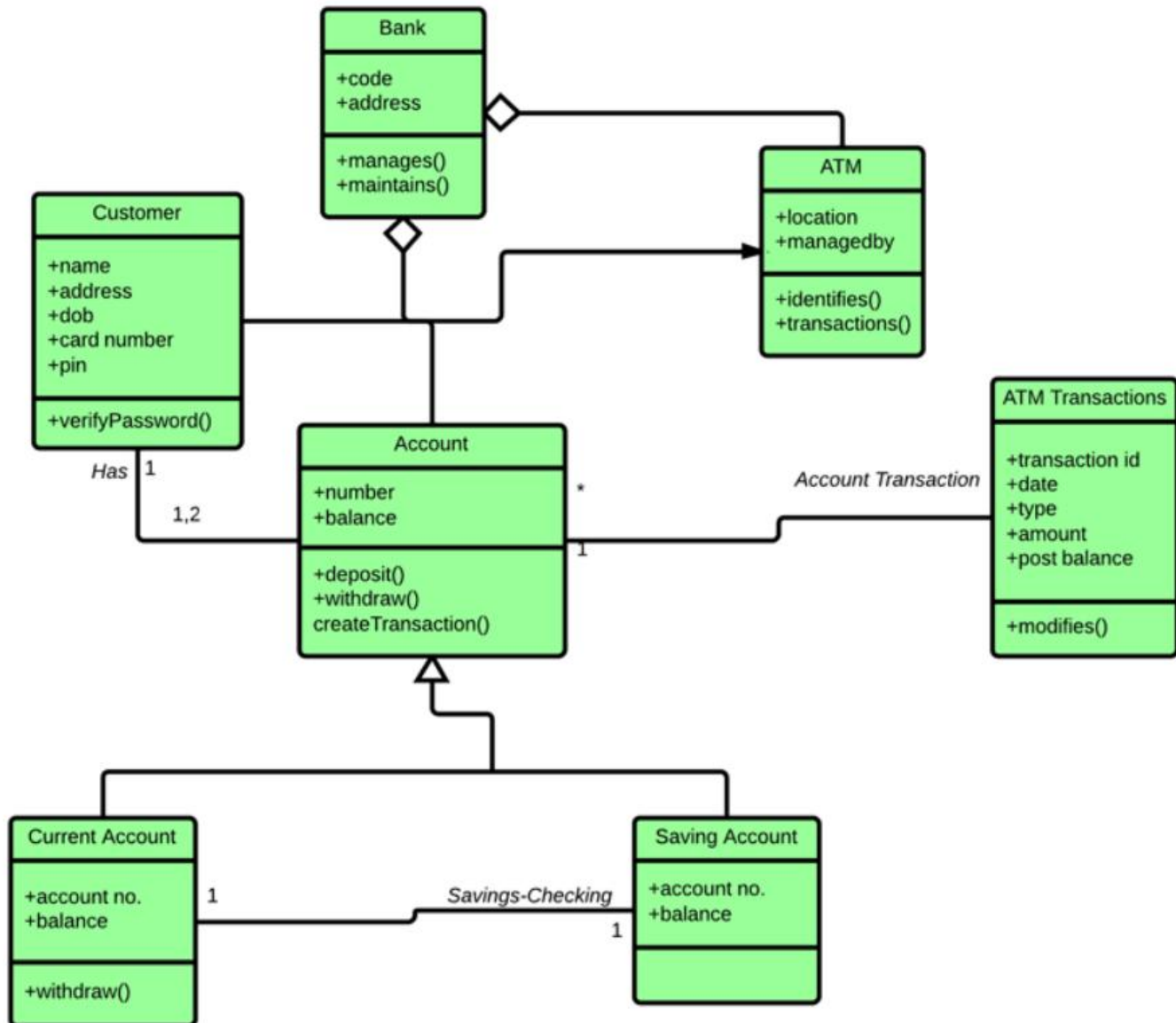
has-a Relationship

- Composition(HAS-A) simply mean the use of instance variables that are references to other objects.
- For example Maruti has Engine, or House has Bathroom.

has-a Relationship

- Use a car as an example. You could logically define a car as being a subclass of vehicle, so it inherits the properties of other vehicles. At the same time, a car has a motor. If you buy a car, you can logically assume that you are buying a motor as well.





has-a Relationship

```
class Office
{
    Department d;
    // any other members
};
```

Example

- Employee class has the reference of Address class as data member

aggregation

```
#include <iostream>
using namespace std;
class Address {
public:
    string city;
    string street;

    Address(string c, string s) {
        city=c;
        street =s; }
    void display() {
        cout << "Address: " << street << ", " << city << endl;
    }
};
```

aggregation

```
class Employee {  
    string name;  
    Address* address; // Aggregation: Address is not owned by Employee  
public:  
    Employee(string n, Address* addr) {  
        name=n;  
        address=addr;}  
  
    void display() {  
        cout << "Employee: " << name << " lives at ";  
        address->display();  
    }  
};
```

aggregation

```
int main() {  
    Address addr("New York", "5th Avenue");  
    Employee emp1("Alice", &addr);  
    Employee emp2("Bob", &addr);  
    emp1.display();  
    emp2.display();  
  
    return 0;  
}
```

Engine composition

- In the case of composition, the Engine is completely encapsulated by the Car. There is no way for the outside world to get a reference to the Engine. The Engine lives and dies with the car.

composition

```
#include <iostream>
using namespace std;
class Engine {
public:
    Engine() {
        cout << "Engine created." << endl;
    }
    void start() {
        cout << "Engine started." << endl;
    }
    ~Engine() {
        cout << "Engine destroyed." << endl;
    }
};
```

composition

```
class Car {
    Engine engine; // Composition: Car "owns" Engine
public:
    Car() {
        cout << "Car created." << endl;
    }
    void drive() {
        engine.start();
        cout << "Car is driving." << endl; }
    ~Car() {
        cout << "Car destroyed." << endl;
    }
};

int main() {
    Car car;
    car.drive();
    return 0;}
```

The following steps happen in order:

1. Engine is constructed first (since it's a member of Car).
2. Car's constructor is executed.

Why does this happen?

When an object of a class is created, all its member objects (like engine inside Car) are created first, before the constructor of the class itself runs.

Composition Rule: The contained objects (member variables) must exist before the containing object (i.e., Car cannot exist without Engine).

Execution Order

The Car class contains an instance of Engine (Engine engine;). Before Car's constructor runs, the compiler first constructs all member objects.

Thus, Engine's constructor runs before Car's constructor.

```
#include <iostream>
using namespace std;
class Engine {
    int horsepower;
public:
    Engine(int hp) {
        horsepower = hp;
        cout << "Engine created with " << horsepower << " HP." << endl; }

    void start() {
        cout << "Engine with " << horsepower << " HP started." << endl;
    }

    ~Engine() {
        cout << "Engine destroyed." << endl;
    }
};
```

```
class Car {
    Engine engine; // Composition: Car "owns" Engine
public:
    Car(int hp) : engine(hp) { // Initialize Engine with horsepower
        cout << "Car created." << endl;
    }

    void drive() {
        engine.start();
        cout << "Car is driving." << endl;
    }

    ~Car() {
        cout << "Car destroyed." << endl;
    }
};

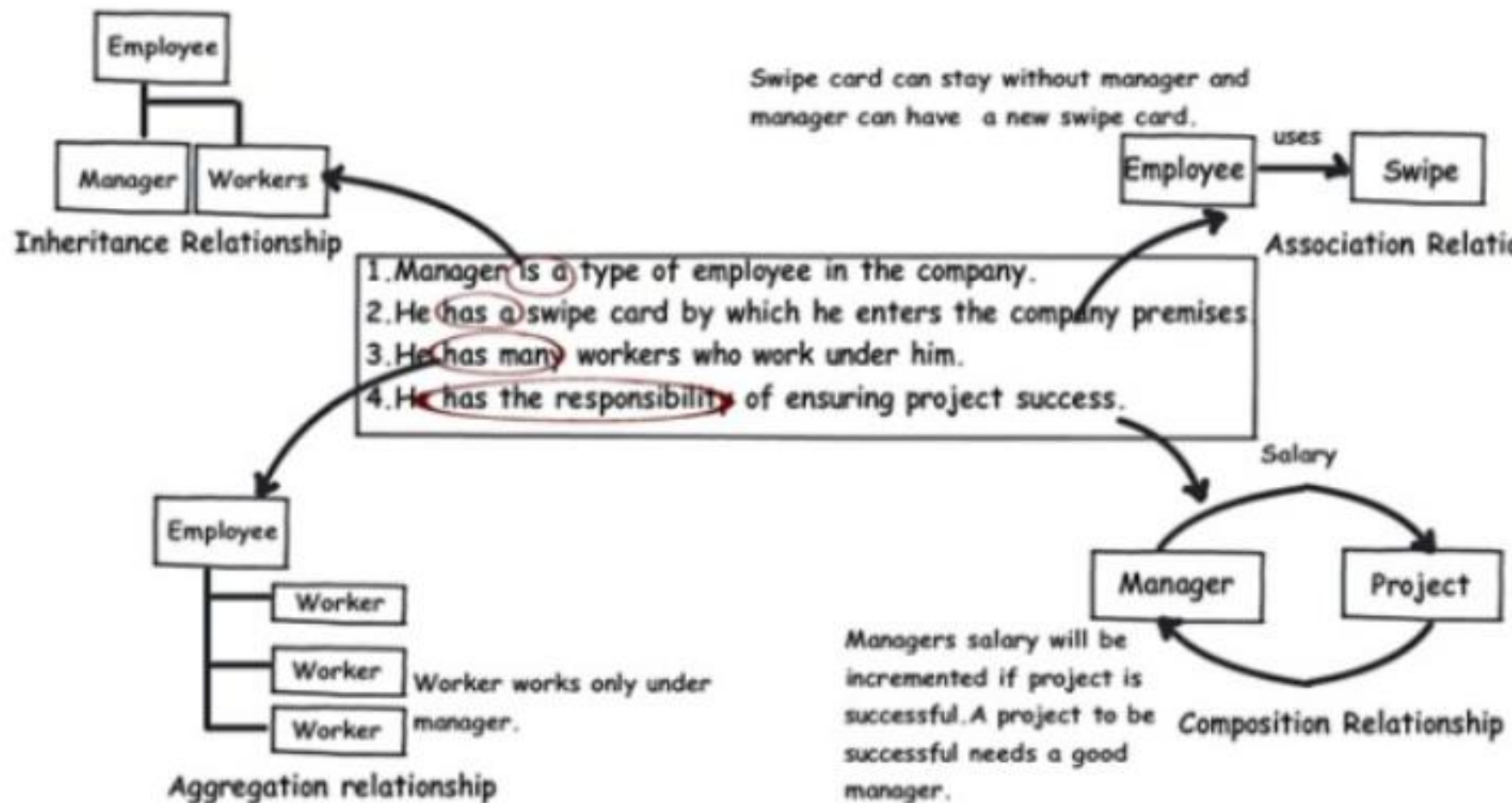
int main() {
    Car car(200); // Pass horsepower to Car, which passes it to Engine
    car.drive();
    return 0;
}
```

Electric Cars

- With aggregation, the Car also performs its functions through an Engine, but the Engine is not always an internal part of the Car. Engines may be swapped, or even completely removed. Not only that, but the outside world can still have a reference to the Engine

Putting things together

Below is a visual representation of how the relationships have emerged from the



Extracting real world relationships from a requirement

1. Manager is an employee of XYZ limited corporation.
2. Manager uses a swipe card to enter XYZ premises.
3. Manager has workers who work under him.
4. Manager has the responsibility of ensuring that the project is successful.
5. Manager's salary will be judged based on project success.

Requirement 1: The IS A relationship

If you look at the first requirement (Manager is an employee of XYZ limited corporation), it's a parent child relationship or inheritance relationship.

The sentence above specifies that Manager is a type of employee
, in other words we will have two classes: parent class Employee, and a child class Manager which will inherit from the Employee class.

Requirement 2: The Using relationship: Association

- Requirement 2 is an interesting requirement (Manager uses a swipe card to enter XYZ premises). In this requirement, the manager object and the swipe card object use each other but they have their own object life time. In other words, they can exist without each other. The most important point in this relationship is that there is no single owner.

```
#include <iostream>
#include <string>
using namespace std;

class SwipeCard {
    string cardID;
public:
    SwipeCard(string id) {
        cardID = id;
    }

    void authorizeAccess() {
        cout << "Access granted using Swipe Card ID: " << cardID << endl;
    }

    string getCardID() const {
        return cardID;
    }
};

class Manager {
    string name;
public:
    Manager(string n) {
        name = n;
    }

    void useSwipeCard(SwipeCard card) {
        cout << "Manager " << name << " is using Swipe Card ID: " << card.getCardID() << endl;
        card.authorizeAccess();
    }
};

int main() {
    SwipeCard card("SC12345");
    Manager manager("John Doe");
    manager.useSwipeCard(card);
    return 0;
}
```

Requirement 3: The Using relationship with Parent: Aggregation

The third requirement from our list (Manager has workers who work under him) denotes the same type of relationship like association but with a difference that one of them is an owner.

So as per the requirement, the Manager object will own Worker objects.

But... the Worker object can have its own life time which is completely disconnected from the Manager object. Looking from a different perspective, it means that if the Manager object is deleted, the Worker object does not die.

```
#include <iostream>
using namespace std;
```

```
class Worker {
    string name;
public:
    Worker(string n) {
        name = n;
    }

    void showWorker() {
        cout << "Worker: " << name << endl;
    }
};
```

```
class Manager {
    string name;
    Worker* workers[3]; // Fixed-size array instead of vector
    int workerCount;
public:
    Manager(string n) {
        name = n;
        workerCount = 0;
    }
};
```

```
void addWorker(Worker* worker) {
    if (workerCount < 3) {
        workers[workerCount++] = worker;
    } else {
        cout << "Cannot add more workers!" << endl;
    }
}

void showWorkers() {
    cout << "Manager: " << name << " has the following workers:\n";
    for (int i = 0; i < workerCount; i++) {
        workers[i]->showWorker();
    }
}

};

int main() {
    Worker w1("Alice");
    Worker w2("Bob");
    Worker w3("Charlie");

    Manager manager("John Doe");
    manager.addWorker(&w1);
    manager.addWorker(&w2);
    manager.addWorker(&w3);

    manager.showWorkers();

    return 0;
}
```

Requirements 4 and 5: The Death relationship: Composition

- The last two requirements are actually logically one. If you read closely, the requirements are as follows:
 1. Manager has the responsibility of ensuring that the project is successful.
 2. Manager's salary will be judged based on project success.
- Below is the conclusion from analyzing the above requirements:
 1. Manager and the project objects are dependent on each other.
 2. The lifetimes of both the objects are the same. In other words, the project will not be successful if the manager is not good, and the manager will not get good increments if the project has issues.

```
#include <iostream>
using namespace std;

class Project {
    string projectName;
    bool isSuccessful;
public:
    Project(string name, bool success) {
        projectName = name;
        isSuccessful = success;
    }

    void showProjectStatus() {
        cout << "Project: " << projectName
            << " | Status: " << (isSuccessful ? "Successful" : "Failed") << endl;
    }

    bool getSuccessStatus() {
        return isSuccessful;
    }
};
```



```

class Manager {
    string name;
    double salary;
    Project project; // Composition: Project is a part of Manager
public:
    Manager(string n, double s, string projectName, bool success)
        : project(projectName, success) { // Temporary object is created first
        name = n;
        salary = s;
    }

    void evaluateSalary() {
        if (project.getSuccessStatus()) {
            salary *= 1.2; // Increase salary by 20% if project is successful
        } else {
            salary *= 0.8; // Decrease salary by 20% if project failed
        }
    }

    void showDetails() {
        cout << "Manager: " << name << " | Salary: $" << salary << endl;
        project.showProjectStatus();
    }
};

int main() {
    // Creating Manager objects (which also creates their respective Projects)
    Manager manager1("John Doe", 5000, "AI Development", true);
    manager1.evaluateSalary();
    manager1.showDetails();

    Manager manager2("Jane Smith", 6000, "Blockchain Security", false);
    manager2.evaluateSalary();
    manager2.showDetails();

    return 0;
}

```

Difference

	Association	Aggregation	Composition
Owner	No owner	Single owner	Single owner
Life time	Have their own lifetime	Have their own lifetime	Owner's life time
Child object	Child objects all are independent	Child objects belong to a single parent	Child objects belong to a single parent

Exercise

Imagine you are designing a student course management system for a university. In this system:

Each **student** can enroll in multiple **courses**.

Each **course** has a unique course ID, name, and number of credits.

A student must be able to view the list of their enrolled courses along with the total credits they have earned from these courses.

Task:

Design a simple class structure in which the **Student** class aggregates **Course** objects.

Write a method in the **Student** class to:

- Add a new course.

- Calculate and return the total number of credits from the enrolled courses.

```
#include <iostream>
#include <string>
using namespace std;

#define MAX_COURSES 10

class Course {
private:
    string courseName;
    int courseID;
    int credits;

public:
    Course(string name = "", int id = 0, int cr = 0) {
        courseName = name;
        courseID = id;
        credits = cr;
    }

    string getCourseName() {
        return courseName;
    }

    int getCredits() {
        return credits;
    }

    void displayCourseInfo() {
        cout << "Course ID: " << courseID
              << ", Name: " << courseName
              << ", Credits: " << credits << endl;
    }
};
```

```
class Student {  
private:  
    string studentName;  
    Course courses[MAX_COURSES];  
    int courseCount;  
  
public:  
    Student(string name) {  
        studentName = name;  
        courseCount = 0;  
    }  
  
    void addCourse(Course &course) {  
        if (courseCount < MAX_COURSES) {  
            courses[courseCount] = course;  
            courseCount++;  
        } else {  
            cout << "Cannot add more courses. Maximum limit reached." << endl;  
        }  
    }  
  
    int calculateTotalCredits() {  
        int totalCredits = 0;  
        for (int i = 0; i < courseCount; i++) {  
            totalCredits += courses[i].getCredits();  
        }  
        return totalCredits;  
    }  
}
```

```
void displayStudentInfo() {  
    cout << "Student Name: " << studentName << endl;  
    cout << "Enrolled Courses:" << endl;  
    for (int i = 0; i < courseCount; i++) {  
        courses[i].displayCourseInfo();  
    }  
    cout << "Total Credits: " << calculateTotalCredits() << endl;  
}  
};
```

```
int main() {  
    Course course1("Data Structures", 101, 4);  
    Course course2("Algorithms", 102, 3);  
    Course course3("Operating Systems", 103, 3);  
  
    Student student("John Doe");  
  
    student.addCourse(course1);  
    student.addCourse(course2);  
    student.addCourse(course3);  
  
    student.displayStudentInfo();  
  
    return 0;  
}
```

Exercise

- A university needs to maintain student academic records. Each student has a **Transcript** that contains detailed information about their academic progress, such as completed courses and grades. The **Transcript** is uniquely owned by the **Student** and cannot exist independently.
- **Task:**
 1. Create a **Student** object with the name "Alex Smith."
 2. Update Alex's transcript to indicate completion of the course "**Operating Systems**" with a grade of **B+**.
 3. Display the student's information along with their updated transcript record.

```
#include <iostream>
#include <string>
using namespace std;

class Transcript {
private:
    string record;

public:
    Transcript() {
        record = "No records available";
    }

    void setRecord(string rec) {
        record = rec;
    }

    void displayRecord() {
        cout << "Transcript Record: " << record << endl;
    }
};
```



```
class Student {
private:
    string studentName;
    Transcript transcript;

public:
    Student(string name) {
        studentName = name;
    }

    void setTranscriptRecord(string record) {
        transcript.setRecord(record);
    }

    void displayStudentInfo() {
        cout << "Student Name: " << studentName << endl;
        transcript.displayRecord();
    }
};

int main() {
    Student student("Jane Doe");
    student.setTranscriptRecord("Completed Data Structures with A grade.");
    student.displayStudentInfo();

    return 0;
}
```



is-a Relationship

- Sometimes, one class *is* an extension of another class

A car *is a* vehicle

Cricket *is a* sport

is-a Relationship

- The extended (or child) class contains all the features of its base (or parent) class, and may additionally have some unique features of its own
- The key idea behind inheritance

Advantage

- It Provides REUSABILITY and MAINTAINABILITY of CODE.

when to use **is a** and
has a relationship?

give some (at least one)
real world example that
implements **is a** and **has a**
relationship

Next Lecture

Inheritance!

